LINCROS
by Sankhya

DOCUMENTAÇÃO TÉCNICA · APACHE NIFI

RepairXmlJson

Processor que corrige JSON e XML quebrados na entrada do fluxo, antes que eles parem a sua integração.

```
{'nome': 'Ana'', "uf": SP,}   →   {"nome":"Ana","uf":"SP"}
<emp>Silva & Filhos</emp>   →   <emp>Silva &amp;amp; Filhos</emp>
"SÃ£o Paulo"   →   "São Paulo"
```

Versão	Compatível com	Data
1.0.0	Apache NiFi 1.23 (Java 8+)	Novembro de 2026

EM RESUMO

Dados sujos não precisam parar o fluxo

Basta uma aspa a mais, um `&` solto ou um acento com encoding errado para um JSON ou XML deixar de ser lido. O fluxo do NiFi falha, a fila trava e alguém precisa corrigir o arquivo à mão. O **RepairXmlJson** fica logo na entrada, corrige automaticamente o que dá para corrigir e registra tudo o que foi feito.

55+
tipos de anomalia corrigidos

0
alterações em documentos já válidos

53 KB
tamanho do pacote, sem dependências

66
testes automatizados, todos passando

Sumário

1 Visão geral
2 Como funciona
3 Instalação
4 Configuração
5 Saídas e atributos
6 Correções suportadas
7 Códigos de correção
8 Exemplos reais
9 Fluxo recomendado
10 Detalhes técnicos
11 Limitações e boas práticas
12 Perguntas frequentes

Para quem é este documento

Quem administra o NiFi
Instalação, atualização e cluster: capítulos 3 e 10.

Quem monta os fluxos
Configuração, saídas e fluxo recomendado: capítulos 4, 5 e 9.

Quem analisa os dados
O que é corrigido e como ler `repair.actions`: capítulos 6, 7 e 8.

Quem decide pelo uso
Visão geral, limitações e boas práticas: capítulos 1 e 11.

Em uma frase

Instale o arquivo `.nar`, coloque o RepairXmlJson logo depois da ingestão, ligue `success` ao seu fluxo e `failure` a uma quarentena. Pronto: arquivos com aspas, espaços ou caracteres errados deixam de travar a integração.

CAPÍTULO 1

Visão geral

O RepairXmlJson é um processor customizado para o Apache NiFi. Ele recebe qualquer FlowFile com JSON ou XML, verifica se o conteúdo está válido e, se não estiver, corrige.

O problema

Em integrações com ERPs, transportadoras, embarcadores e sistemas legados, é comum receber arquivos com pequenos defeitos:

- **Aspas** simples no lugar de duplas, aspas repetidas, aspas sem escape no meio do texto ou aspas "curvas" copiadas do Word e do Excel.
- **Espaços** sobrando nas pontas dos valores, dentro das tags XML ou invisíveis (NBSP, zero-width).
- **Caracteres especiais**: `&` e `<` soltos no XML, caracteres de controle, BOM e acentos quebrados (`SÃ£o`).
- **Estrutura**: vírgula faltando ou sobrando, tags que não fecham, chaves sem aspas, vírgula decimal (`1,5`).

Processors como `EvaluateJsonPath` , `JoltTransformJSON` , `ConvertRecord` e `EvaluateXPath` rejeitam esses arquivos, e o fluxo para.

A solução

Válidos passam intactos

Um documento que já está correto sai byte a byte igual ao que entrou. Só os quebrados são tocados.

Corrige na entrada

Detecta sozinho se é JSON ou XML e aplica as correções antes de qualquer outro processor ler o conteúdo.

Tudo rastreável

Cada correção vira um código no atributo `repair.actions` , com contagem. Dá para auditar e medir a qualidade dos dados.

Nunca perde dados

Se não conseguir garantir um resultado válido, o FlowFile vai para `failure` com o conteúdo original e o motivo.

Leve e sem dependências

Pacote de 53 KB, só com a biblioteca padrão do Java. Não conflita com nada que já está instalado no NiFi.

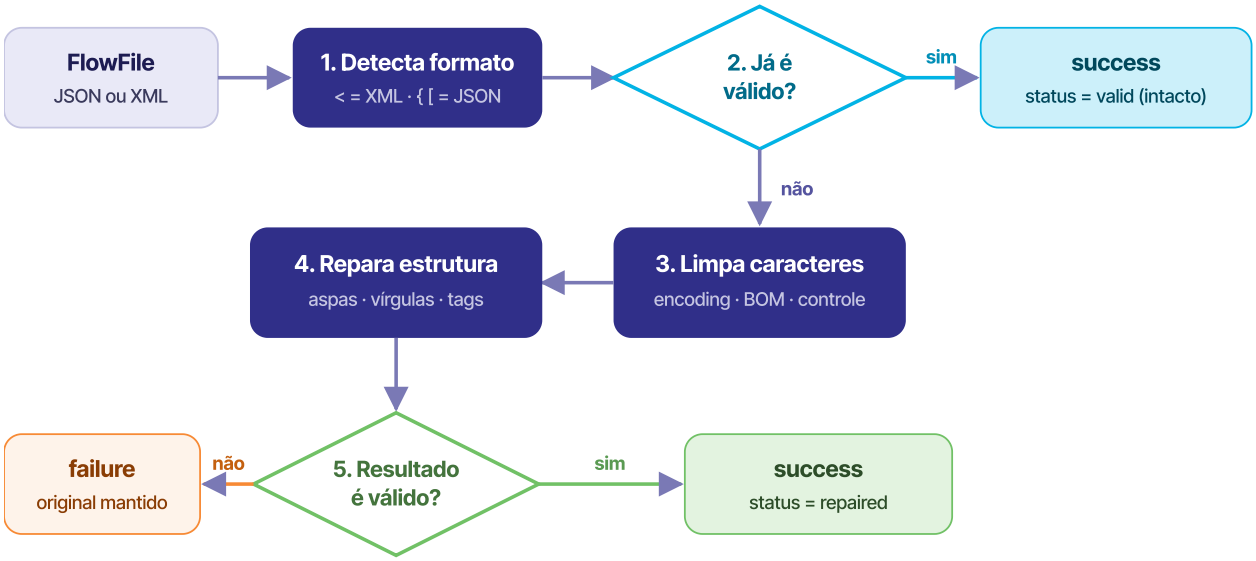
Testado de verdade

66 testes automatizados e validação de ponta a ponta em um NiFi 1.23.2 real, com carregamento sem reiniciar.

CAPÍTULO 2

Como funciona

Cada FlowFile passa por até cinco etapas. A decisão mais importante vem logo no início: se o documento já é válido, nada é alterado.



As etapas

1. **Detecta o formato.** Olha o primeiro caractere significativo: `<` é XML; `{` ou `[` é JSON. O formato também pode ser fixado na configuração.
2. **Verifica se já é válido.** Usa um validador estrito: o padrão RFC 8259 para JSON e o parser XML do Java para XML. Se for válido e o modo for "Somente inválidos", o FlowFile segue sem nenhuma alteração.
3. **Limpa os caracteres.** Lê o arquivo de forma tolerante (UTF-8, ou Windows-1252 se houver bytes inválidos), corrige acentos quebrados, remove BOM, caracteres invisíveis e de controle e troca espaços especiais por espaço comum.
4. **Repara a estrutura.** Um leitor tolerante entende o documento quebrado (aspas, vírgulas, tags, atributos, entidades) e reescreve tudo no formato correto.
5. **Valida o resultado.** O documento corrigido passa de novo pelo validador estrito. Só vai para `success` se estiver 100% válido. Senão, vai para `failure` com o conteúdo original.

> **Como ele decide se uma aspa é "de fechamento" ou "do meio do texto"?**
> Olhando o que vem depois. Em `"Tubo 3" de aço"`, a aspa depois do 3 é seguida por `de`, que não faz sentido estruturalmente, então ela é tratada como parte do texto. A aspa depois de `aço` é seguida por `,` ou `}`, então fecha o valor. A mesma ideia vale para atributos XML.

CAPÍTULO 3

Instalação

A instalação é copiar um arquivo. No NiFi 1.23 não é preciso reiniciar o serviço.

Requisitos

Item	Requisito
Apache NiFi	1.23.x (funciona em versões 1.x com carregamento automático de NAR, a partir da 1.9)
Java do NiFi	8, 11, 17 ou 21 (o pacote é compilado para Java 8)
Arquivo	`nifi-repair-nar-1.0.0.nar` (53 KB)
Para compilar (opcional)	JDK 8+ e Maven 3.6+

Passo a passo

1. **Copie o arquivo para a pasta de carregamento automático.**

```
cp nifi-repair-nar-1.0.0.nar $NIFI_HOME/extensions/
```

O NiFi verifica essa pasta a cada poucos segundos e carrega o processor sozinho. A pasta é definida em `nifi.nar.library.autoload.directory` (padrão: `./extensions`).

2. **Confira no log.** Em alguns segundos devem aparecer estas linhas em `logs/nifi-app.log`:

```
NarAutoLoaderTask Found ./extensions/nifi-repair-nar-1.0.0.nar in auto-load directory
StandardExtensionDiscoveringManager Loaded extensions for br.com.lincros.nifi:nifi-repair-nar:1.0.0
```

3. **Adicione ao fluxo.** No canvas, arraste o ícone de **Processor**, digite **RepairXmlJson** no filtro e conecte o processor logo depois da ingestão.

4. **Ligue as saídas.** Conecte `success` ao restante do fluxo e `failure` a uma área de quarentena (por exemplo, `PutFile` ou `PutS3Object`).

Cenários específicos

Cluster

Copie o `.nar` para a pasta `extensions/` de **todos os nós**. Se um nó não tiver o processor, o fluxo fica inválido nesse nó.

Docker

```
docker cp nifi-repair-nar-1.0.0.nar <container>:/opt/nifi/nifi-current/extensions/
```

Para manter o processor quando o container for recriado, monte `extensions/` como volume.

Sem carregamento automático

Copie para `$NIFI_HOME/lib/` e reinicie o NiFi.

CAPÍTULO 4

Configuração

Todas as propriedades têm um valor padrão seguro. Na maioria dos fluxos, basta adicionar o processor e conectar.

Propriedade	Padrão	O que faz
Formato de entrada `input-format`	Automático	**Automático** detecta pelo primeiro caractere. **JSON** ou **XML** fixam o formato, o que é útil quando o arquivo começa com lixo.
Modo `repair-mode`	Somente inválidos	**Somente inválidos**: documentos válidos passam sem alteração. **Sempre normalizar**: todo documento é limpo, inclusive os válidos. Use esse modo para corrigir acentos quebrados e espaços em arquivos que estão estruturalmente corretos.
Remover espaços nas pontas `trim-whitespace`	true	Apara espaços no início e no fim de chaves e valores JSON, valores de atributos XML e texto de elementos XML sem filhos. Exemplo: `" Ana "` vira `"Ana"`.
Corrigir acentuação quebrada `fix-mojibake`	true	Corrige texto UTF-8 que foi lido com a codificação errada: `SÃ£o` vira `São` e `AÃ§Ãºcar` vira `Açúcar`.
JSON indentado `pretty-json`	false	Saída JSON indentada, para leitura humana, ou compacta, em uma linha.
Elemento agrupador XML `xml-wrapper-element`	root	Nome do elemento usado para agrupar quando o arquivo tem mais de um XML (mais de uma raiz).

> **Qual modo escolher?**
>
> Comece com **Somente inválidos**: é o modo mais conservador e não mexe no que já funciona. Mude para **Sempre normalizar** se a origem dos dados manda acentos quebrados ou espaços sobrando mesmo em arquivos válidos.

Agendamento e desempenho

- O processor suporta **batching** e não tem efeitos colaterais, então pode rodar com várias tarefas em paralelo (*Concurrent Tasks*).
- Cada FlowFile é carregado inteiro na memória. Para arquivos de até algumas dezenas de MB, o custo é baixo.
- Documentos válidos no modo "Somente inválidos" só passam pela validação, sem reescrita. É o caminho mais rápido.

CAPÍTULO 5

Saídas e atributos

Relacionamentos

Saída	Quando	Conteúdo
success	O documento já era válido ou foi corrigido com sucesso.	Original (se válido) ou corrigido, sempre em UTF-8.
failure	Formato não reconhecido, documento vazio, ou o resultado não ficou válido.	Sempre o **original**, sem nenhuma alteração.

Status

repair.status	Significado	Saída
valid	Já era válido. Nada foi alterado.	success
repaired	Era inválido e foi corrigido.	success
normalized	Era válido, mas foi limpo (só no modo "Sempre normalizar").	success
failed	Não foi possível corrigir com segurança.	failure

Atributos gravados

Atributo	Descrição	Exemplo
`repair.status`	Resultado do processamento (veja a tabela acima).	`repaired`
`repair.format`	Formato detectado ou configurado.	`json` ou `xml`
`repair.actions`	Correções aplicadas, com a contagem de cada uma. Os códigos estão no capítulo 7.	`aspas-extras-removidas(2),virgula-extra-removida(1)`
`repair.action.count`	Total de correções aplicadas.	`3`
`repair.error`	Motivo da falha. Só existe em `failure`.	`Formato não reconhecido...`
`mime.type`	Tipo do conteúdo, útil para processors seguintes.	`application/json`

Rastreabilidade (Provenance)

Quando o conteúdo é alterado, o processor registra um evento **CONTENT_MODIFIED** no Data Provenance, com a lista de correções nos detalhes. Na interface do NiFi, dá para ver o conteúdo de antes e de depois de cada FlowFile corrigido. Documentos válidos não geram evento, porque nada mudou.

CAPÍTULO 6

Correções suportadas

Exemplos de antes e depois para cada tipo de anomalia. Todos estão cobertos pelos testes automatizados.

JSON

Problema	Antes	Depois
Aspas simples	`{'nome': 'Ana'}`	`{"nome":"Ana"}`
Aspa extra no fim do valor	`{"nome": "Ana"", "id": 1}`	`{"nome":"Ana","id":1}`
Aspa simples sobrando	`{"nome": "Ana"', "id": 1}`	`{"nome":"Ana","id":1}`
Aspa extra no início	`{"nome": ""Ana", "x": 1}`	`{"nome":"Ana","x":1}`
Aspas dobradas	`{""nome"": ""Ana""}`	`{"nome":"Ana"}`
Aspa interna sem escape	`{"d": "Tubo 3" de aço"}`	`{"d":"Tubo 3\" de aço"}`
Apóstrofo em aspas simples	`{'obs': 'caixa d'água'}`	`{"obs":"caixa d'água"}`
Aspas do Word/Excel	`{“nome”: “Ana”}`	`{"nome":"Ana"}`
Vírgula faltando	`{"a": 1 "b": 2}`	`{"a":1,"b":2}`
Vírgula sobrando	`[1, 2,, 3,]`	`[1,2,3]`
Chave sem aspas	`{nome: "Ana"}`	`{"nome":"Ana"}`
Comentários	`{"a": 1 // obs` … `/* x */}`	`{"a":1}`
Estrutura não fechada	`{"a": [1, 2, {"b": "x"`	`{"a":[1,2,{"b":"x"}]}`
Valor ausente	`{"a": , "b": 1}`	`{"a":null,"b":1}`
Vírgula decimal	`{"peso": 1,5, "qtd": 2}`	`{"peso":1.5,"qtd":2}`
Zero à esquerda (CEP, NF)	`{cep: 01310100}`	`{"cep":"01310100"}`
Texto sem aspas	`{"uf": SP}`	`{"uf":"SP"}`
Literais Python/JavaScript	`{'a': None, 'b': True}`	`{"a":null,"b":true}`
Caminho Windows (escape inválido)	`{'p': 'C:\pasta'}`	`{"p":"C:\\pasta"}`
Quebra de linha crua na string	`"linha1⏎linha2"`	`"linha1\nlinha2"`
Vários JSON no arquivo	`{"a":1}⏎{"a":2}`	`[{"a":1},{"a":2}]`
Lixo antes ou depois	`resposta: {"a": 1} fim`	`{"a":1}`

> **Por que o CEP vira texto?**
> Em JSON, `01310100` não é um número válido. Converter para número faria perder o zero à esquerda (`1310100`), o que estraga CEPs, números de nota e códigos. Por isso o valor é mantido como texto.

XML

Problema	Antes	Depois
`&` solto	`<a>Silva & Filhos</a>`	`<a>Silva & Filhos</a>`
`<` solto no texto	`<a>x < 5</a>`	`<a>x < 5</a>`
Aspa interna em atributo	`<i d="Tubo 3" aço"/>`	`<i d="Tubo 3" aço"/>`
Aspas extras em atributo	`<i id=""10""/>`	`<i id="10"/>`
Atributo sem aspas	`<i id=10/>`	`<i id="10"/>`
Atributo sem valor	`<i ativo/>`	`<i ativo="ativo"/>`
Atributo duplicado	`<i id="1" id="2"/>`	`<i id="1"/>`
Espaço dentro da tag	`< nome >Ana</ nome >`	`<nome>Ana</nome>`
Tag não fechada	`<a><b>1<c>2</a>`	`<a><b>1<c>2</c></b></a>`
Fechamento sem abertura	`<a>1</b></a>`	`<a>1</a>`
Maiúsculas/minúsculas	`<Nome>Ana</nome>`	`<Nome>Ana</Nome>`
Entidades HTML	`São Paulo`	`São Paulo`
Espaço/BOM antes do `<?xml`	`␣␣<?xml version="1.0"?>`	`<?xml version="1.0" encoding="UTF-8"?>`
Várias raízes	`<a>1</a><a>2</a>`	`<root><a>1</a><a>2</a></root>`
Declarações repetidas	dois `<?xml?>` concatenados	uma declaração, raízes agrupadas
Prefixo sem namespace	`<ns:a>...</ns:a>`	`<ns:a xmlns:ns="urn:nao-declarado:ns">`
Encoding ISO-8859-1	`encoding="ISO-8859-1"`	conteúdo e declaração em UTF-8

Comentários (`<!--` `-->`), blocos CDATA e instruções de processamento são preservados.

Os dois formatos

Problema	Antes	Depois
Espaços nas pontas	`" Ana "`	`"Ana"`
Acentuação quebrada	`SÃ£o Paulo` · `AÃ§Ã°car`	`São Paulo` · `Açúcar`
Arquivo em Windows-1252	bytes Latin-1 (`S\xe3o`)	UTF-8 (`São`)
BOM, invisíveis e controle	`\uFEFF` · `\u200B` · `\u0001`	removidos
Espaços especiais (NBSP)	`\u00A0`	espaço comum

CAPÍTULO 7

Códigos de correção

Cada correção aplicada aparece em `repair.actions` com um código e a quantidade de vezes que ocorreu, por exemplo `espacos-aparados(3)`.

Caracteres e encoding (JSON e XML)

Código	Significado
`utf8-invalido-lido-como-cp1252`	O arquivo tinha bytes que não são UTF-8 e foi lido como Windows-1252.
`encoding-corrigido`	Acento quebrado corrigido (`Ã§` virou `ç`).
`bom-ou-caractere-invisivel-removido`	BOM ou caractere de largura zero removido.
`espaco-unicode-normalizado`	NBSP ou outro espaço especial trocado por espaço comum.
`caractere-controle-removido`	Caractere de controle invisível removido.
`charset-desconhecido-ignorado`	O XML declarava um encoding inexistente; foi lido como UTF-8.
`espacos-aparados`	Espaços removidos das pontas de um valor, chave, atributo ou texto.
`aspas-extras-removidas`	Aspa repetida ou sobrando foi descartada.
`aspas-internas-escapadas`	Aspa dupla no meio do texto foi escapada (`\"` ou `"`).
`aspas-tipograficas-convertidas`	Aspas curvas (“ ” ‘ ’) ou crase usadas como delimitador foram trocadas por aspas retas.
`lixo-inicial-removido`	Texto antes do início do documento foi removido.

JSON

Código	Significado
`aspas-simples-convertidas`	Aspas simples usadas como delimitador foram trocadas por aspas duplas.
`chave-sem-aspas-corrigida`	Chave sem aspas recebeu aspas.
`texto-sem-aspas-corrigido`	Valor de texto sem aspas recebeu aspas.
`virgula-ausente-adicionada`	Faltava vírgula entre dois itens.
`virgula-extra-removida`	Vírgula sobrando (no fim ou repetida) foi removida.
`virgula-decimal-corrigida`	Número com vírgula decimal (`1,5`) virou `1.5` .
`dois-pontos-ausente-adicionado`	Faltava `:` entre chave e valor.
`igual-trocado-por-dois-pontos`	`chave = valor` virou `chave: valor` .
`objeto-nao-fechado` · `array-nao-fechado`	Faltava `}` ou `]` no final.
`string-nao-fechada`	Texto sem aspa de fechamento foi encerrado.
`fechamento-trocado-corrigido`	`]` usado no lugar de `}` , ou o contrário.
`valor-ausente-preenchido-null`	Chave sem valor recebeu `null` .
`literal-normalizado`	`True` , `None` , `undefined` , `NaN` viraram `true` / `null` .

Código	Significado
`numero-normalizado`	Número fora do padrão (`+5` , `.5` , `5.`) foi ajustado.
`zero-a-esquerda-mantido-como-texto`	Número com zero à esquerda virou texto para não perder dígitos.
`escape-invalido-corrigido`	Barra invertida inválida (ex.: `C:\pasta`) foi mantida como texto.
`comentario-removido`	Comentário `//` ou `/* */` removido.
`chave-duplicada-ultima-mantida`	Chave repetida no mesmo objeto; o último valor foi mantido.
`caractere-invalido-removido`	Caractere solto fora de lugar foi removido.
`lixo-final-removido`	Texto depois do fim do documento foi removido.
`multiplas-raizes-agrupadas-em-array`	Vários JSON no mesmo arquivo foram agrupados em uma lista.

XML

Código	Significado
`e-comercial-escapado`	`&` solto virou `&` .
`menor-que-escapado`	`<` solto no texto virou `<` .
`entidade-html-convertida`	Entidade HTML (`ã` , ` ` ...) virou o caractere real.
`entidade-invalida-removida`	Referência numérica a um caractere proibido em XML foi removida.
`atributo-sem-aspas-corrigido`	Valor de atributo sem aspas recebeu aspas.
`atributo-sem-valor-corrigido`	Atributo sem valor (`ativo`) virou `ativo="ativo"` .
`atributo-duplicado-removido`	Atributo repetido na mesma tag; o primeiro foi mantido.
`aspa-de-atributo-nao-fechada`	Valor de atributo sem aspa de fechamento foi encerrado.
`espaco-dentro-da-tag-removido`	`< nome >` virou `<nome>` .
`caractere-invalido-na-tag-removido`	Caractere solto dentro de uma tag foi removido.
`maior-que-ausente-adicionado`	Tag sem o `>` de fechamento foi completada.
`tag-nao-fechada`	Elemento aberto e nunca fechado foi fechado.
`tag-de-fechamento-orfa-removida`	Tag de fechamento sem abertura correspondente foi removida.
`maiusculas-minusculas-da-tag-corrigidas`	Fechamento com maiúsculas/minúsculas diferentes da abertura foi ajustado.
`espaco-antes-da-declaracao-removido`	Espaço ou quebra de linha antes do `<?xml` foi removido.
`declaracao-encoding-ajustada-utf8`	Declaração de encoding diferente de UTF-8 foi ajustada.
`declaracao-xml-extra-removida`	`<?xml?>` repetido no meio do arquivo foi removido.
`texto-fora-da-raiz-removido`	Texto solto fora do elemento principal foi removido.
`multiplas-raizes-agrupadas`	Mais de um elemento principal foi agrupado no elemento agrupador.
`namespace-declarado`	Prefixo usado sem declaração recebeu um namespace.
`comentario-corrigido` · `comentario-nao-fechado`	Comentário com `--` interno ou sem `-->` foi ajustado.
`cdata-nao-fechado`	Bloco CDATA sem `]]>` foi fechado.

CAPÍTULO 8

Exemplos reais

Resultados do teste de ponta a ponta em um **Apache NiFi 1.23.2** real, com o fluxo GetFile → RepairXmlJson → PutFile. Todas as saídas foram conferidas com leitores de JSON e XML independentes.

Aspas simples, aspa extra e acento quebrado

repaired · 6 correções

ENTRADA

```
{'cliente': 'Transportes Silva & Filhos'', 'cidade': 'SÃ£o Paulo', "obs": "caixa d'água"}
```

SAÍDA

```
{"cliente":"Transportes Silva & Filhos","cidade":"São Paulo","obs":"caixa d'água"}
```

Aspa interna, vírgula decimal, CEP e vírgula sobrando

repaired · 6 correções

ENTRADA

```
{"produto": "Tubo 3" de aço", "peso": 1,5, "cep": 01310100, ativo: True,}
```

SAÍDA

```
{"produto":"Tubo 3\" de aço","peso":1.5,"cep":"01310100","ativo":true}
```

BOM, caractere de controle, espaços e vírgula faltando

repaired · 5 correções

ENTRADA

```
[BOM]{ "nome" : " Ana ",[0x01] "fone": "(11)[NBSP]9999-0000"↵ "uf": "SP"↵}
```

SAÍDA

```
{"nome":"Ana","fone":"(11) 9999-0000","uf":"SP"}
```

Arquivo salvo em Windows-1252

repaired · 1 correção

ENTRADA (BYTES)

```
{"cidade": "S\xe3o Jos\xe9", "obs": "a\xe7\xfacar"}
```

SAÍDA

```
{"cidade":"São José","obs":"açúcar"}
```

XML com vários problemas ao mesmo tempo

repaired · 14 correções

ENTRADA

```
␣␣<?xml version="1.0"?><nota><emit nome="Silva "Express" Ltda" cnpj=12345678000199>Silva & Filhos</em
it>< obs >peso < 10kg   S&atilde;o Paulo</ obs ><item qtd=""10"">Caixa</nota>
```

SAÍDA

```
<?xml version="1.0" encoding="UTF-8"?><nota><emit nome="Silva &quot;Express&quot; Ltda" cnpj="12345678
000199">Silva &amp; Filhos</emit><obs>peso &lt; 10kg São Paulo</obs><item qtd="10">Caixa</item></nota>
```

JSON válido

valid

ENTRADA = SAÍDA (INTACTO)

```
{"valido": true, "itens": [1, 2, 3]}
```

Conteúdo irreconhecível

failed

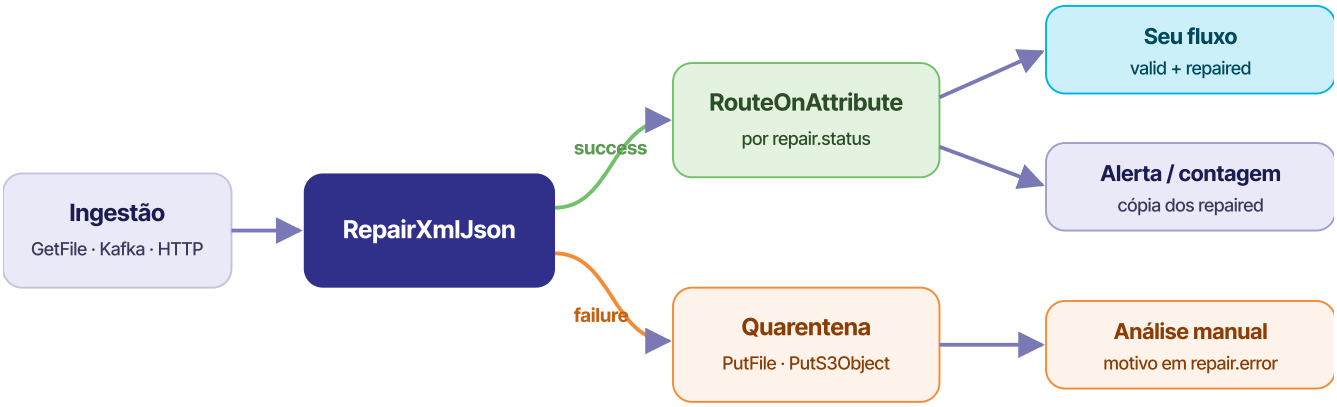
VAI PARA FAILURE, ORIGINAL

```
isso nao e json nem xml
```

CAPÍTULO 9

Fluxo recomendado

Coloque o processor logo depois da ingestão e antes de qualquer processor que leia o conteúdo. Use os atributos para separar, alertar e medir.



Expressões úteis

Propriedades dinâmicas para o `RouteOnAttribute` (Expression Language do NiFi):

```
# só os que foram corrigidos (para alerta ou auditoria)
corrigidos      = ${repair.status:equals('repaired')}

# muitas correções num mesmo arquivo: vale investigar a origem
muito_sujo      = ${repair.action.count:gt(10)}

# houve troca de encoding: o sistema de origem está gravando em Latin-1
encoding_errado = ${repair.actions:contains('utf8-invalido-lido-como-cp1252')}
```

Painel de qualidade de dados

Os atributos `repair.status` e `repair.actions` permitem medir, por sistema de origem ou parceiro, quantos arquivos chegam quebrados e quais problemas se repetem. Uma forma simples de montar isso:

1. Use `AttributesToJSON` para gerar um registro com `filename`, `repair.status`, `repair.actions` e a origem do arquivo.
2. Grave esses registros num banco (`PutDatabaseRecord`) ou num índice de logs.
3. Monte um painel com a taxa de arquivos corrigidos por origem e os códigos mais frequentes. Assim, dá para levar dados concretos ao parceiro que precisa ajustar a integração.

CAPÍTULO 10

Detalhes técnicos

Arquitetura

Classe	Responsabilidade
`RepairXmlJson`	O processor do NiFi: lê as propriedades e o conteúdo, chama o `Repairer`, grava atributos e envia para `success` ou `failure`.
`Repairer`	Orquestra o processo: detecta o formato, valida, chama a limpeza e o reparo e decide o status.
`TextCleaner`	Decodificação tolerante (UTF-8 com fallback para Windows-1252), mojibake, BOM, invisíveis, espaços especiais e caracteres de controle.
`JsonRepairer`	Leitor JSON tolerante, com as heurísticas de aspas, vírgulas e estrutura, e a escrita do JSON válido. Preserva números exatamente como vieram.
`XmlRepairer`	Varredura tolerante de XML: tags, atributos, entidades, pilha de elementos, namespaces e validação final com o parser do Java.
`StrictJsonValidator` · `RepairLog` · `RepairOptions`	Validador JSON estrito (RFC 8259); registro das correções com contagem; opções vindas das propriedades.

Compatibilidade e segurança

- **Bytecode Java 8**: roda no NiFi 1.23 com Java 8, 11, 17 ou 21.
- **Sem dependências externas**: usa só a biblioteca padrão do Java. A `nifi-api` é fornecida pelo próprio NiFi.
- **XML seguro**: a validação desativa entidades externas e o carregamento de DTDs remotos, o que protege contra ataques XXE.
- **Aninhamento extremo**: JSON com mais de 1000 níveis não derruba o processor.

Compilar a partir do código

```
git clone https://github.com/jaapa07/Claude.git
cd Claude/nifi-repair-bundle
mvn clean package
# resultado: nifi-repair-nar/target/nifi-repair-nar-1.0.0.nar
# módulos: nifi-repair-processors (código e testes) · nifi-repair-nar (pacote)
```

Testes

Conjunto	Testes	O que cobre
`JsonRepairTest`	34	Todas as correções de JSON, encoding, modos e saída indentada.
`XmlRepairTest`	25	Todas as correções de XML, incluindo uma NF-e realista com vários problemas.
`RepairXmlJsonProcessorTest`	7	O processor no simulador oficial do NiFi: rotas, atributos, provenance e lotes.
Ponta a ponta	8 arquivos	NiFi 1.23.2 real, com carregamento do NAR sem reiniciar e saídas conferidas por leitores independentes.

CAPÍTULO 11

Limitações e boas práticas

Limitações conhecidas

> **A correção é heurística**
> Quando um documento está quebrado, às vezes existe mais de uma interpretação possível. O processor escolhe a mais provável, mas pode errar em casos ambíguos. Exemplo: uma aspa sem escape no meio do texto seguida de `, "`, como em `"diz "oi", "tchau""`.

- **Memória**: o conteúdo é carregado inteiro. Para arquivos muito grandes (centenas de MB), divida antes com `SplitRecord`, `SplitText` ou `SplitXml`, se possível.
- **Encoding de saída**: sempre UTF-8. Se um sistema seguinte exige ISO-8859-1, use `ConvertCharacterSet` depois.
- **Namespaces fictícios**: prefixos XML sem declaração recebem `urn:nao-declarado:<prefixo>`. Se você usa XPath com namespace, ajuste a consulta.
- **Modo "Sempre normalizar" e DTD**: entidades personalizadas definidas em DTD interno são tratadas como texto.
- **Espaços no XML**: quando o modo de reparo é acionado, espaços entre elementos fora da raiz são descartados, e o NBSP vira espaço comum.

Boas práticas

- **Monitore os corrigidos.** Mande uma cópia dos `repaired` para auditoria nas primeiras semanas e confira se as correções fazem sentido para os seus dados.
- **Nunca descarte o failure.** Ligue sempre a saída `failure` a uma quarentena. São arquivos que precisam de atenção humana.
- **Corrija na origem.** Use `repair.actions` para mostrar aos parceiros quais problemas se repetem. O processor resolve o sintoma; a causa está em quem gera o arquivo.
- **Comece conservador.** Use o modo "Somente inválidos" e só mude para "Sempre normalizar" quando precisar.
- **Fixe o formato quando souber.** Se uma fila só recebe JSON, configure o formato como JSON. Isso evita detecção errada quando o arquivo começa com lixo.

CAPÍTULO 12

Perguntas frequentes

O processor não aparece na lista do NiFi. O que fazer?

Confira se o arquivo está em `$NIFI_HOME/extensions/` (ou na pasta definida em `nifi.nar.library.autoload.directory`) e procure por `nifi-repair-nar` em `logs/nifi-app.log`. Em cluster, o arquivo precisa estar em todos os nós. Se o carregamento automático estiver desligado, copie para `lib/` e reinicie.

Como atualizo para uma nova versão?

Copie o novo `.nar` (com outro número de versão) para `extensions/`. Depois, no processor, use **Change Version** para trocar de versão, e só então remova o arquivo antigo.

Ele altera arquivos que já estão corretos?

Não, no modo padrão ("Somente inválidos"). O conteúdo sai byte a byte igual e o status fica `valid`. Só o modo "Sempre normalizar" mexe em documentos válidos.

Posso perder dados?

O processor nunca descarta o original: se não conseguir garantir um resultado válido, envia o arquivo sem alterações para `failure`. Nas correções, algumas informações podem mudar de propósito, como espaços nas pontas, caracteres de controle e chaves duplicadas. Todas aparecem em `repair.actions` e no Provenance.

Números grandes perdem precisão?

Não. Os números são copiados exatamente como vieram, sem conversão para `double`. Por exemplo, `12345678901234567890.123456789` sai idêntico.

Funciona no NiFi 2.x?

Esta versão foi feita e testada para o NiFi 1.23. O NiFi 2.x exige Java 21 e tem mudanças de API, então o código precisaria ser recompilado com as dependências do 2.x.

Como desligo só uma das correções?

Hoje é possível desligar o aparo de espaços e a correção de acentos pelas propriedades. As demais correções são parte do reparo estrutural, porque sem elas o documento continuaria inválido.

Qual o impacto no desempenho?

Para documentos válidos, só há o custo da validação. Para documentos quebrados, o reparo é feito em uma única passada pelo texto. O processor suporta lotes e várias tarefas em paralelo.

Onde está o código?

No repositório `jaapa07/Claude`, branch `feature/nifi-repair-xml-json`, pasta `nifi-repair-bundle/`.



Logística não precisa ser complicada para ser eficiente.

RepairXmlJson 1.0.0 · Documentação técnica · Novembro de 2026